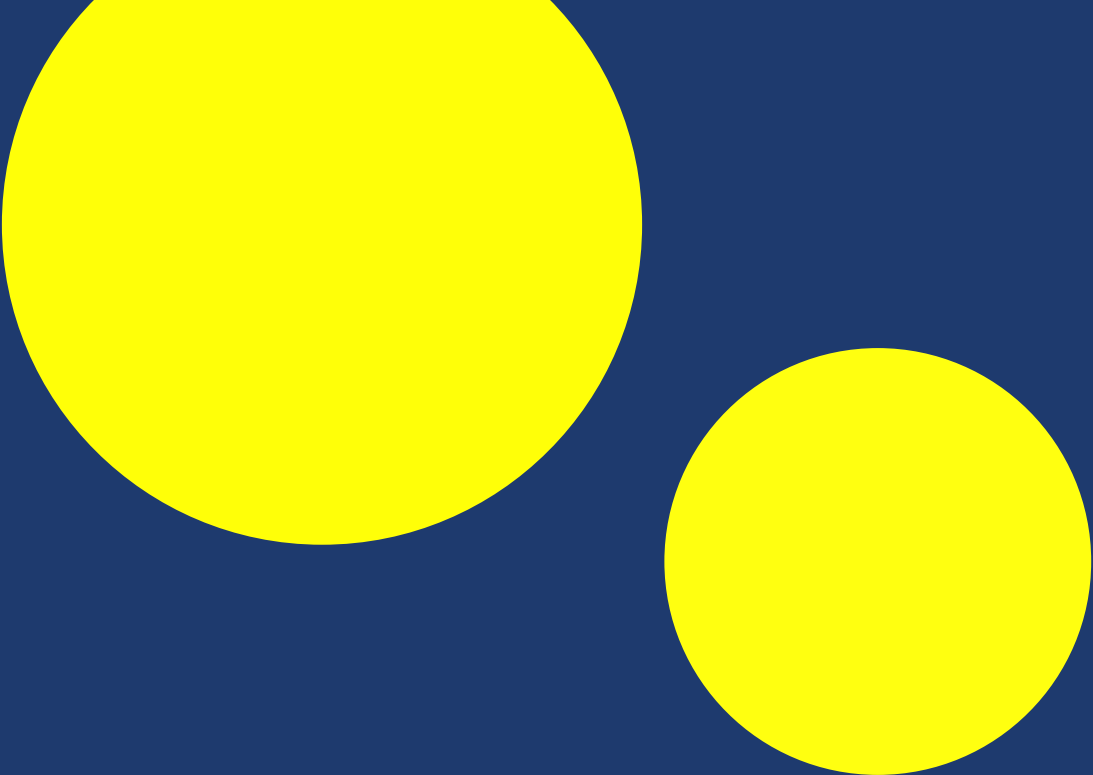




Smart Job & Freelance Platform — Architecture Plan

Complete Model Design · Service Architecture · Workflow · Database Plan

Angular 19 + Bootstrap 5 + JSON Server → Spring Boot Migration

Version 1.0 — May 2026 — Senior Architecture Review

Table of Contents

1	Project Overview & Tech Stack	3
2	Architecture Rules & Principles	4
3	Role System & Access Control	5
4	Folder Structure & Service Architecture	6
5	Shared Models	7
6	Auth & User Domain — All Models	8
7	Job Portal Domain — All Models	11
8	CV / Resume Domain — All Models	14
9	Freelance Marketplace Domain — All Models	17
10	Orders & Disputes Domain — All Models	20
11	Payments Domain — All Models	22
12	Chat Domain — All Model	24
13	Service Architecture — 2-Layer Pattern	25
14	API Service Method Reference	27
15	Workflow Diagrams	29
16	Database Design — MySQL Tables	31
17	JSON Server — db.json Reference	33
18	Environment & Migration Plan	34
19	Build Roadmap — 7 Phases	35
20	Risk Analysis	36

1. Project Overview & Tech Stack

Smart Job & Freelance Platform

This document is the complete architectural reference for a unified Job Portal + Freelance Marketplace platform targeting the Bangladesh market. It combines features of BdJobs/Indeed (job portal) with Fiverr (freelance marketplace) into one ecosystem.

What we are building

A single platform where one account can hold multiple roles:

- Post jobs as an Employer
- Apply for jobs as a Jobseeker
- Offer services as a Freelancer
- Hire freelancers as a Client
- Manage everything as an Admin

Guest users can browse jobs, gigs, and public profiles without logging in.

Tech stack

Frontend: Angular 19+, Standalone Components

Language: TypeScript strict mode

Styling: Bootstrap 5 + bootstrap-icons + SCSS

State: RxJS + Angular Signals

Forms: Reactive Forms only — no ngModel

Fake backend: JSON Server (db.json, port 3000)

Future backend: Spring Boot + MySQL + JWT

Migration: Change one URL in environment.ts

2. Architecture Rules & Principles

Non-negotiable rules for the entire project

These rules are locked in for the entire project lifetime. Violating any of them creates Spring Boot migration problems.

API Layer rule	Component → Feature Service → API Service → HttpClient Components NEVER call HttpClient directly. API Services contain HttpClient ONLY. Feature Services contain business logic ONLY.
Model naming	Every interface is named with Model suffix: UserModel, JobModel. Every file is named: user.model.ts, job.model.ts One interface per file. No type aliases. No enums.
ID fields	id?: string — optional because absent before save. Foreign key ids (userId, jobId, orderId) are required string fields. IDs are generated by json-server on POST. UUIDs in Spring Boot.
Address rule	NEVER store address as a plain string. Always use AddressModel: divisionId, districtId, upazilaId, postCode, addressLine. This maps to normalized Bangladesh address data.
Forms rule	Reactive Forms ONLY. No ngModel. No template-driven forms. All form validation is reactive. All form submission goes through a service.
TypeScript strict	No "any" types. No non-null assertions (!). No implicit any. Use optional chaining (?.) safely. Every variable must be typed.
State management	No NgRx unless absolutely necessary. Use Angular Signals for reactive state. Use RxJS for async data streams. Keep state in services, not in components.
Environment	ALL API URLs come from environment.ts. environment.ts has ONE property: apiUrl. Switching to Spring Boot = change one string. Nothing else.

3. Role System & Access Control

Single account — multiple roles

One user account can hold multiple roles simultaneously. A person can be both a Jobseeker and a Freelancer. Role checks are performed in route guards and in component logic.

Role	What they can do	Guard
JOBSEEKER	Browse jobs, apply, track applications, build CV, set job alerts	authGuard + roleGuard
EMPLOYER	Post jobs, review applications, manage hiring pipeline, schedule interviews	authGuard + roleGuard
FREELANCER	Create gigs, manage packages, accept orders, deliver work, earn money	authGuard + roleGuard
CLIENT	Browse gigs, place orders, communicate, release payment, rate freelancers	authGuard + roleGuard
ADMIN	Full platform access: users, jobs, gigs, disputes, analytics, commissions	authGuard + adminGuard

Guest access (unauthenticated)

Action	Details
Browse all jobs	Search and filter public job listings
Browse all gigs	Search and filter public gig listings
View public profiles	Employer and freelancer public pages
View job detail	Full job description — no apply button
View gig detail	Full gig page with packages — no order button

4. Folder Structure & Service Architecture

Feature-first organization — Spring Boot compatible

Every domain owns its own `models/`, `services/`, and `components/` folders. Nothing from one domain imports from another domain except through `shared/` or `core/`.

```
src/app/
|-- core/
|   |-- interceptors/    jwt.interceptor.ts  error.interceptor.ts
|   |-- guards/         auth.guard.ts  admin.guard.ts  role.guard.ts
|   |-- services/       base-api.service.ts  token.service.ts
|
|-- shared/
|   |-- models/         address.model.ts
|   |-- components/     navbar/  sidebar/  footer/  star-rating/
|                       status-badge/  otp-input/  address-selector/
|
|-- features/
|   |-- auth/
|   |   |-- models/     otp.model.ts  password-reset.model.ts
|   |   |-- services/   auth-api.service.ts  auth.service.ts
|   |   |-- components/ login/  register/  verify-otp/  forgot-password/  reset-password/
|   |
|   |-- users/
|   |   |-- models/     user.model.ts  notification.model.ts  rating.model.ts  report.model.ts
|   |   |-- services/   user-api.service.ts  user.service.ts  [one pair per model]
|   |   |-- components/ profile/  notifications/
|   |
|   |-- jobs/
|   |   |-- models/     job.model.ts  job-application.model.ts  job-alert.model.ts
|   |   |               hiring-pipeline.model.ts  interview-round.model.ts
|   |   |-- services/   [api + feature service per model]
|   |   |-- components/ job-list/  job-detail/  post-job/  my-applications/  pipeline/
|   |
|   |-- cv/
|   |   |-- models/     resume.model.ts  education.model.ts  experience.model.ts
|   |   |               certification.model.ts  project.model.ts
|   |   |-- services/   [api + feature service per model]
|   |   |-- components/ cv-builder/  cv-preview/  education-form/  experience-form/
|   |
|   |-- gigs/
|   |   |-- models/     gig.model.ts  gig-package.model.ts  gig-category.model.ts
|   |   |-- services/   [api + feature service per model]
|   |   |-- components/ gig-list/  gig-detail/  create-gig/  manage-gigs/
|   |
|   |-- orders/
|   |   |-- models/     order.model.ts  dispute.model.ts
|   |   |-- services/   [api + feature service per model]
|   |   |-- components/ order-list/  order-detail/  dispute-form/
|   |
|   |-- payments/
|   |   |-- models/     wallet.model.ts  transaction.model.ts
|   |   |-- services/   [api + feature service per model]
|   |   |-- components/ wallet/  transaction-history/  withdraw/
|   |
|   |-- chat/
|   |   |-- models/     message.model.ts
|   |   |-- services/   message-api.service.ts  message.service.ts
|   |   |-- components/ chat-window/  message-list/  message-input/
|   |
|   |-- admin/
|   |   |-- components/ dashboard/  user-management/  disputes/  analytics/
```

```
|  
|-- layouts/  
|   |-- public-layout/   navbar + footer for guest pages  
|   |-- dashboard-layout/ sidebar + header for authenticated users  
|   |-- admin-layout/    sidebar + header for admin only  
|  
`-- environments/  
    |-- environment.ts    apiUrl: 'http://localhost:3000' <-- THE migration switch
```

5. Shared Models

Used across all domains

AddressModel

File: src/app/shared/models/address.model.ts | **MySQL table:** Embedded in users and jobs tables (not a separate table)

Normalized Bangladesh address structure. Used by UserModel and JobModel. NEVER store address as a plain city/area/district string. Each ID references a lookup table of BD divisions, districts, and upazilas.

Field	Type	Role	Purpose
divisionId	string	FK	References bd_divisions lookup table
districtId	string	FK	References bd_districts lookup table
upazilaId	string	FK	References bd_upazilas lookup table
postCode	string		Bangladesh postal code (4-digit)
addressLine	string		Street address, house/flat number, area

6. Auth & User Domain

UserModel · OtpModel · PasswordResetModel · NotificationModel · RatingModel · ReportModel

UserModel

File: src/app/features/users/models/user.model.ts | **MySQL table:** users

Core account record. Every other model references this via userId FK. One user can hold multiple roles. passwordHash is NEVER displayed in UI — backend sets it. Address uses normalized AddressModel, not plain strings.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated by json-server / UUID in Spring Boot
fullName	string		User full name as displayed across the platform
email	string		Unique login email — checked before registration
phone	string		Bangladesh mobile number — used for OTP
passwordHash	string		Bcrypt hash in Spring Boot — plain text in JSON Server
roles	string[]		Array of roles: 'JOBSEEKER' 'EMPLOYER' 'FREELANCER' 'CLIENT' 'ADMIN'
profilePicture	string null	opt	URL to profile image — null until uploaded
address	AddressModel null	opt	Normalized Bangladesh address — null until provided
status	string		Account status: 'ACTIVE' 'SUSPENDED' 'PENDING_VERIFICATION'
isEmailVerified	boolean		True after OTP email verification completes
isPhoneVerified	boolean		True after OTP SMS verification completes
createdAt	string		ISO 8601 datetime string — set on registration
updatedAt	string		ISO 8601 datetime string — set on every update

OtpModel

File: src/app/features/auth/models/otp.model.ts | **MySQL table:** otps

Stores OTP codes for email/phone verification and password reset. Each OTP has a purpose so the same table handles all verification flows. isUsed prevents replay attacks. expiresAt enforces time limit (10 minutes).

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
userId	string	FK	References users.id — whose OTP this is
code	string		6-digit numeric OTP code
purpose	string		'EMAIL_VERIFY' 'PHONE_VERIFY' 'PASSWORD_RESET'
expiresAt	string		ISO datetime — 10 minutes after creation
isUsed	boolean		True after code is consumed — prevents reuse
createdAt	string		ISO datetime when OTP was generated

PasswordResetModel

File: src/app/features/auth/models/password-reset.model.ts | **MySQL table:** password_resets

Tracks password reset tokens. Used by forgot-password and reset-password flows. Token is sent via email/SMS. isUsed prevents token reuse.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
userId	string	FK	References users.id
token	string		Secure random token — sent to user email/phone
expiresAt	string		ISO datetime — 1 hour after creation
isUsed	boolean		True after password is successfully reset
createdAt	string		ISO datetime when reset was requested

NotificationModel

File: src/app/features/users/models/notification.model.ts | **MySQL table:** notifications

In-app notification for any platform event. Every significant action creates a notification for the relevant user. referenceId links to the specific job, order, or message that triggered it.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
userId	string	FK	References users.id — recipient of this notification
type	string		'JOB_APPLICATION' 'APPLICATION_STATUS' 'ORDER_PLACED' 'ORDER_DELIVERED' 'MESSAGE_RECEIVED' 'PAYMENT_RECEIVED' 'DISPUTE_OPENED'
title	string		Short notification heading — shown in notification bell
body	string		Full notification text — shown in notification panel
isRead	boolean		False until user opens notification — drives unread count badge
referenceId	string null	opt	ID of related entity (orderId, jobId, messageId)
createdAt	string		ISO datetime when notification was created

RatingModel

File: src/app/features/users/models/rating.model.ts | **MySQL table:** ratings

Review left after an order completes. Both client and freelancer can rate each other. targetType determines who is being rated. score drives average rating display on profiles.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
orderId	string	FK	References orders.id — rating is always tied to a completed order
reviewerId	string	FK	References users.id — who wrote this review

Field	Type	Role	Purpose
revieweeId	string	FK	References users.id — who is being reviewed
targetType	string		'FREELANCER' 'CLIENT' 'EMPLOYER'
score	number		1 to 5 — integer star rating
comment	string		Written review text — displayed on profile
createdAt	string		ISO datetime when review was submitted

ReportModel

File: src/app/features/users/models/report.model.ts | **MySQL table:** reports

User-submitted report against another user, job, or gig for violations. Admin reviews and takes action. status tracks the moderation workflow.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
reporterId	string	FK	References users.id — who filed this report
targetId	string		ID of the reported entity (userId, jobId, or gigId)
targetType	string		'USER' 'JOB' 'GIG'
reason	string		'SPAM' 'FRAUD' 'INAPPROPRIATE' 'FAKE' 'OTHER'
description	string		Detailed explanation of the violation
status	string		'PENDING' 'REVIEWED' 'ACTION_TAKEN' 'DISMISSED'
adminNote	string null	opt	Admin resolution note — null until reviewed
createdAt	string		ISO datetime when report was filed

7. Job Portal Domain

JobModel · JobApplicationModel · JobAlertModel · HiringPipelineModel · InterviewRoundModel

JobModel

File: src/app/features/jobs/models/job.model.ts | **MySQL table:** jobs

A job listing posted by an employer. Central record for the job portal — applications, pipeline, alerts, and interviews all reference this. skills is a JSON array in MySQL. address uses normalized AddressModel.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
employerId	string	FK	References users.id — must have EMPLOYER role
title	string		Job title displayed in listings and search results
description	string		Full HTML-safe job description
requirements	string		Candidate requirements — education, experience, skills
responsibilities	string		Day-to-day duties of the role
jobType	string		'FULL_TIME' 'PART_TIME' 'CONTRACT' 'INTERNSHIP'
experienceLevel	string		'ENTRY' 'MID' 'SENIOR' 'EXECUTIVE'
salaryMin	number null	opt	Minimum salary in BDT — null if not disclosed
salaryMax	number null	opt	Maximum salary in BDT — null if not disclosed
isSalaryNegotiable	boolean		True shows Negotiable tag on listing
skills	string[]		Required skills — stored as JSON array in MySQL
address	AddressModel null	opt	Normalized job location — null for remote
deadline	string		ISO date string — application deadline
status	string		'DRAFT' 'ACTIVE' 'CLOSED' 'ARCHIVED'
totalApplications	number		Counter — incremented when application is submitted
createdAt	string		ISO datetime when job was posted
updatedAt	string		ISO datetime of last edit

JobApplicationModel

File: src/app/features/jobs/models/job-application.model.ts | **MySQL table:** job_applications

Created when a jobseeker applies to a job. Tracks the full lifecycle from application to hired/rejected. resumeId links to the CV snapshot used at time of application.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
jobId	string	FK	References jobs.id
applicantId	string	FK	References users.id — must have JOBSEEKER role
resumeId	string null	FK	References resumes.id — null if applied without CV
coverLetter	string		Custom cover letter text for this application

Field	Type	Role	Purpose
status	string		'APPLIED' 'SHORTLISTED' 'INTERVIEW_SCHEDULED' 'REJECTED' 'HIRED' 'WITHDRAWN'
appliedAt	string		ISO datetime when application was submitted
updatedAt	string		ISO datetime of last status change

JobAlertModel

File: src/app/features/jobs/models/job-alert.model.ts | **MySQL table:** job_alerts

Saved job search preferences for a jobseeker. The system checks new job postings against active alerts and sends notifications. All filter fields are optional — a user can alert on keyword alone.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
userId	string	FK	References users.id — alert owner
keyword	string		Job title or keyword to match against new postings
jobType	string null	opt	Filter by job type — null means any type
experienceLevel	string null	opt	Filter by level — null means any level
divisionId	string null	opt	Filter by division — null means anywhere in BD
isActive	boolean		False to pause without deleting the alert
createdAt	string		ISO datetime when alert was created

HiringPipelineModel

File: src/app/features/jobs/models/hiring-pipeline.model.ts | **MySQL table:** hiring_pipeline

Tracks where each applicant is in the employer hiring funnel. One record per applicant per job. stage is updated as employer moves the candidate through the pipeline.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
jobId	string	FK	References jobs.id
applicantId	string	FK	References users.id
applicationId	string	FK	References job_applications.id
stage	string		'APPLIED' 'SCREENING' 'INTERVIEW' 'OFFER' 'HIRED' 'REJECTED'
updatedAt	string		ISO datetime of last stage change

InterviewRoundModel

File: src/app/features/jobs/models/interview-round.model.ts | **MySQL table:** interview_rounds

One scheduled interview round for an application. An application can have multiple rounds (phone → technical → HR). outcome and notes are filled in after the interview completes.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
applicationId	string	FK	References job_applications.id
round	number		Round number — 1 for first interview, 2 for second, etc.
type	string		'PHONE' 'VIDEO' 'ONSITE' 'TECHNICAL' 'HR'
scheduledAt	string		ISO datetime of scheduled interview
location	string null	opt	Physical address for onsite — null for remote
meetingLink	string null	opt	Video call URL — null for onsite
outcome	string null	opt	'PASSED' 'FAILED' 'RESCHEDULED' — null until decided
notes	string null	opt	Interviewer notes — null until after interview
createdAt	string		ISO datetime when interview was scheduled

8. CV / Resume Domain

ResumeModel · EducationModel · ExperienceModel · CertificationModel · ProjectModel

ResumeModel

File: src/app/features/cv/models/resume.model.ts | **MySQL table:** resumes

Root CV record for a jobseeker. One user can have multiple resumes. isDefault marks which resume is used when applying to jobs without choosing. Sub-sections (education, experience, certifications, projects) are fetched separately.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
userId	string	FK	References users.id — resume owner
headline	string		e.g. "Senior Software Engineer" — shown below name on CV
summary	string		Professional summary / career objective paragraph
careerObjective	string		Objective statement for specific role types
skills	string[]		Skill tags — stored as JSON array
availableFor	string		'FULL_TIME' 'PART_TIME' 'REMOTE' 'FREELANCE' 'INTERNSHIP'
expectedSalary	number		Expected monthly salary in BDT
presentSalary	number		Current monthly salary in BDT — 0 if fresher
careerLevel	string		'ENTRY' 'MID' 'SENIOR' 'TOP_LEVEL'
address	AddressModel null	opt	Current location using normalized Bangladesh address
permanentAddress	string null	opt	Free-text permanent address
githubUrl	string null	opt	GitHub profile URL
portfolioUrl	string null	opt	Portfolio website URL
linkedinUrl	string null	opt	LinkedIn profile URL
cvVisibility	string		'PUBLIC' 'PRIVATE' 'RECRUITERS_ONLY'
profileCompletion	number		Percentage 0–100 — computed by ResumeService
isDefault	boolean		True for the resume used in quick-apply
createdAt	string		ISO datetime
updatedAt	string		ISO datetime of last edit

EducationModel

File: src/app/features/cv/models/education.model.ts | **MySQL table:** educations

One education entry belonging to a resume. Separate table in MySQL — fetched when building the CV view. endYear is null if currently enrolled.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
resumeId	string	FK	References resumes.id

Field	Type	Role	Purpose
institution	string		School, college, or university name
degree	string		e.g. BSc, HSC, SSC, Masters
fieldOfStudy	string		e.g. Computer Science, Business Administration
board	string null	opt	Board name for SSC/HSC — null for university
startYear	number		4-digit year enrollment started
endYear	number null	opt	4-digit year — null if currently enrolled
result	string null	opt	GPA or grade result — e.g. 5.00, A+
achievements	string null	opt	Awards or notable achievements during study

ExperienceModel

File: src/app/features/cv/models/experience.model.ts | **MySQL table:** experiences

One work experience entry belonging to a resume. endDate is null when isCurrentJob is true. responsibilities is displayed as bullet points on the CV.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
resumeId	string	FK	References resumes.id
companyName	string		Employer company name
jobTitle	string		Position / designation held
department	string null	opt	Department within the company
startDate	string		ISO date string — month and year
endDate	string null	opt	ISO date string — null if currently working here
isCurrentJob	boolean		True drives "Present" label on CV instead of end date
responsibilities	string		Key responsibilities — supports newline-separated bullets
achievements	string null	opt	Notable achievements in this role

CertificationModel

File: src/app/features/cv/models/certification.model.ts | **MySQL table:** certifications

One certification or training entry belonging to a resume. expiryDate null means it does not expire. credentialUrl links to the verifiable certificate.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
resumeId	string	FK	References resumes.id
name	string		Certification name — e.g. AWS Solutions Architect
issuingOrganization	string		Issuer — e.g. Amazon, Coursera, Google
issueDate	string		ISO date string when certification was earned
expiryDate	string null	opt	ISO date string — null if non-expiring

Field	Type	Role	Purpose
credentialId	string null	opt	Certificate ID number for verification
credentialUrl	string null	opt	URL to verify the certificate online

ProjectModel

File: src/app/features/cv/models/project.model.ts | **MySQL table:** projects

One portfolio project entry belonging to a resume. Technologies is a JSON array. Both githubUrl and liveUrl are optional but at least one should be provided.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
resumeId	string	FK	References resumes.id
projectName	string		Project title — displayed on CV
description	string		What the project does and your role in it
technologies	string[]		Tech stack used — stored as JSON array
githubUrl	string null	opt	GitHub repository URL
liveUrl	string null	opt	Live deployment URL
startDate	string null	opt	ISO date string when project started
endDate	string null	opt	ISO date string when project completed

9. Freelance Marketplace Domain

GigCategoryModel · GigModel · GigPackageModel

GigCategoryModel

File: src/app/features/gigs/models/gig-category.model.ts | **MySQL table:** gig_categories

Lookup table for gig categories. parentId enables sub-categories. e.g. "Design" (parent) → "Logo Design" (child). slug is URL-friendly version for routing.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
name	string		Display name — e.g. Web Development, Graphic Design
slug	string		URL slug — e.g. web-development, graphic-design
parentId	string null	opt	References gig_categories.id — null for top-level
icon	string null	opt	Bootstrap icon class — e.g. bi-code-slash

GigModel

File: src/app/features/gigs/models/gig.model.ts | **MySQL table:** gigs

A freelance service listing. Contains up to 3 packages (Basic/Standard/Premium). averageRating and totalOrders are computed fields maintained by the backend. tags is a JSON array used for search and discovery.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
freelancerId	string	FK	References users.id — must have FREELANCER role
categoryId	string	FK	References gig_categories.id
title	string		Gig title — displayed in search and listing
description	string		Full gig description — what you will deliver
tags	string[]		Search tags — stored as JSON array
coverImage	string null	opt	Main gig thumbnail URL
images	string[]		Additional gallery images — JSON array of URLs
status	string		'ACTIVE' 'PAUSED' 'PENDING_REVIEW' 'REJECTED'
averageRating	number		Computed average from ratings table — default 0
totalOrders	number		Completed order count — incremented on order completion
createdAt	string		ISO datetime
updatedAt	string		ISO datetime of last edit

GigPackageModel

File: src/app/features/gigs/models/gig-package.model.ts | **MySQL table:** gig_packages

One pricing tier belonging to a gig. A gig has exactly 3 packages: BASIC, STANDARD, PREMIUM. Clients choose a package when placing an order. OrderModel references packageId to capture the exact terms at time of order.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
gigId	string	FK	References gigs.id
name	string		'BASIC' 'STANDARD' 'PREMIUM'
description	string		What is included in this package
price	number		Price in BDT — displayed on gig page
deliveryDays	number		Number of days to complete — used to compute deadline
revisions	number		Number of revision rounds included — -1 for unlimited
features	string[]		Feature list — JSON array — displayed as checkmarks on gig page

10. Orders & Disputes Domain

OrderModel · DisputeModel

OrderModel

File: src/app/features/orders/models/order.model.ts | **MySQL table:** orders

Created when a client purchases a gig package. Central record that chat, payments, ratings, and disputes all reference. platformFee and freelancerEarning are calculated at order creation and locked.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
gigId	string	FK	References gigs.id
clientId	string	FK	References users.id — must have CLIENT role
freelancerId	string	FK	References users.id — must have FREELANCER role
packageId	string	FK	References gig_packages.id — locks terms at order time
requirements	string		Client's instructions — what they need delivered
price	number		Total price paid by client in BDT
platformFee	number		Platform commission (%) — deducted from freelancer earning
freelancerEarning	number		Net amount credited to freelancer wallet on completion
status	string		'PENDING' 'IN_PROGRESS' 'DELIVERED' 'REVISION_REQUESTED' 'COMPLETED' 'CANCELLED' 'DISPUTED'
deliveryDeadline	string		ISO datetime — computed from order date + package deliveryDays
deliveredAt	string null	opt	ISO datetime when freelancer clicked Deliver — null until delivered
completedAt	string null	opt	ISO datetime when client accepted delivery — null until complete
revisionsUsed	number		Counter of revision requests used so far
createdAt	string		ISO datetime when order was placed
updatedAt	string		ISO datetime of last status change

DisputeModel

File: src/app/features/orders/models/dispute.model.ts | **MySQL table:** disputes

Raised by client or freelancer when an order goes wrong. Admin reviews evidence and resolves. resolvedAt is set when admin closes the dispute.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
orderId	string	FK	References orders.id — one dispute per order
raisedById	string	FK	References users.id — who opened the dispute
reason	string		'NO_DELIVERY' 'POOR_QUALITY' 'NOT_AS_DESCRIBED' 'COMMUNICATION' 'OTHER'

Field	Type	Role	Purpose
description	string		Detailed explanation of the issue
status	string		'OPEN' 'UNDER_REVIEW' 'RESOLVED' 'CLOSED'
adminNote	string null	opt	Admin's resolution decision — null until resolved
resolvedAt	string null	opt	ISO datetime when admin resolved — null until closed
createdAt	string		ISO datetime when dispute was opened

11. Payments Domain

WalletModel · TransactionModel

WalletModel

File: src/app/features/payments/models/wallet.model.ts | **MySQL table:** wallets

One wallet per user. balance is available for withdrawal. escrowBalance is locked money held for active orders — released to freelancer on completion or returned to client on cancellation.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
userId	string	FK	References users.id — one wallet per user
balance	number		Available balance in BDT — can be withdrawn
escrowBalance	number		Locked balance in BDT — held for in-progress orders
currency	string		Always 'BDT' for Bangladesh Taka
updatedAt	string		ISO datetime of last balance change

TransactionModel

File: src/app/features/payments/models/transaction.model.ts | **MySQL table:** transactions

Every money movement is recorded as an immutable transaction. This is the full audit trail. referenceId links to the order or payment that caused this transaction.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
walletId	string	FK	References wallets.id
type	string		'DEPOSIT' 'WITHDRAWAL' 'ORDER_PAYMENT' 'ORDER_REFUND' 'PLATFORM_FEE' 'FREELANCER_EARNING' 'ESCROW_LOCK' 'ESCROW_RELEASE'
amount	number		Amount in BDT — always positive
referenceId	string null	opt	Related orderId or paymentId — null for manual deposits
description	string		Human-readable description for transaction history
createdAt	string		ISO datetime — immutable record

12. Chat Domain

MessageModel

MessageModel

File: src/app/features/chat/models/message.model.ts | **MySQL table:** messages

Every message is scoped to an order. There is no free-form chat — all communication happens inside an active order context. This design maps naturally to Spring Boot WebSocket rooms keyed by orderId.

Field	Type	Role	Purpose
id?	string	PK	Auto-generated
orderId	string	FK	References orders.id — all messages belong to an order
senderId	string	FK	References users.id — message author
receiverId	string	FK	References users.id — message recipient
content	string		Message text — plain text or markdown
attachmentUrl	string null	opt	File attachment URL — null if text-only message
attachmentType	string null	opt	'IMAGE' 'PDF' 'ZIP' 'OTHER' — null if no attachment
isRead	boolean		False until recipient opens the chat — drives unread badge
sentAt	string		ISO datetime when message was sent

13. Service Architecture — 2-Layer Pattern

The most important architectural rule in this project

For every model there are exactly TWO services. The API service handles HTTP. The feature service handles business logic. Components call the feature service ONLY. Never the API service directly.

Layer 1 — API Service (JobApiService)

Contains ONLY HttpClient calls. No business logic. No conditional checks. No localStorage. Just HTTP. Returns Observables typed to the model.

```
@Injectable({ providedIn: 'root' })
export class JobApiService {
  private url = `${environment.apiUrl}/jobs`;
  constructor(private http: HttpClient) {}

  getAll(): Observable<JobModel[]> {
    return this.http.get<JobModel[]>(this.url);
  }
  getById(id: string): Observable<JobModel> {
    return this.http.get<JobModel>(`${this.url}/${id}`);
  }
  save(job: JobModel): Observable<JobModel> {
    return this.http.post<JobModel>(this.url, job);
  }
  update(id: string, job: JobModel): Observable<JobModel> {
    return this.http.put<JobModel>(`${this.url}/${id}`, job);
  }
  delete(id: string): Observable<void> {
    return this.http.delete<void>(`${this.url}/${id}`);
  }
  // Additional filter methods
  findByEmployerId(employerId: string): Observable<JobModel[]> {
    return this.http.get<JobModel[]>(`${this.url}?employerId=${employerId}`);
  }
  findByStatus(status: string): Observable<JobModel[]> {
    return this.http.get<JobModel[]>(`${this.url}?status=${status}`);
  }
  searchByTitle(keyword: string): Observable<JobModel[]> {
    return this.http.get<JobModel[]>(`${this.url}?title_like=${keyword}`);
  }
}
```

Layer 2 — Feature Service (JobService)

Contains ONLY business logic. Calls JobApiService — NEVER HttpClient. This is what components inject and call.

```
@Injectable({ providedIn: 'root' })
export class JobService {
  constructor(private api: JobApiService) {}

  getAll(): Observable<JobModel[]> {
    return this.api.getAll();
  }
  getById(id: string): Observable<JobModel> {
    return this.api.getById(id);
  }
}
```

```
save(job: JobModel): Observable<JobModel> {
    return this.api.save(job);
}
update(id: string, job: JobModel): Observable<JobModel> {
    return this.api.update(id, job);
}
delete(id: string): Observable<void> {
    return this.api.delete(id);
}
// Business logic methods
getActiveJobs(): Observable<JobModel[]> {
    return this.api.findByStatus('ACTIVE');
}
getJobsByEmployer(employerId: string): Observable<JobModel[]> {
    return this.api.findByEmployerId(employerId);
}
searchJobs(keyword: string): Observable<JobModel[]> {
    return this.api.searchByTitle(keyword);
}
isDeadlinePassed(job: JobModel): boolean {
    return new Date(job.deadline) < new Date();
}
}
```

In a Component

```
export class JobListComponent {
    private jobService = inject(JobService); // CORRECT
    // private jobApiService = inject(JobApiService); // WRONG – never do this

    jobs = signal<JobModel[]>([]);

    ngOnInit(): void {
        this.jobService.getActiveJobs().subscribe(jobs => {
            this.jobs.set(jobs);
        });
    }
}
```

14. API Service Method Reference

5 standard methods + additional filter methods per service

Every API service has the same 5 standard methods. Additional filter methods are specific to each domain. The table below lists all services and their domain-specific methods.

API Service	Endpoint	Standard 5 Methods	Additional Methods
UserApiService	/users	getAll() getById() save() update() delete()	findByEmail(email) findByRole(role) findByStatus(status) searchByName(name)
OtpApiService	/otps	getAll() getById() save() update() delete()	findById(userId) findByPurpose(purpose)
NotificationApiService	/notifications	getAll() getById() save() update() delete()	findById(userId) findUnread(userId) markAllRead(userId)
RatingApiService	/ratings	getAll() getById() save() update() delete()	findByRevieweeId(id) findByOrderId(orderId) findByTargetType(type)
ReportApiService	/reports	getAll() getById() save() update() delete()	findByStatus(status) findByTargetId(targetId)
JobApiService	/jobs	getAll() getById() save() update() delete()	findByEmployerId(id) findByStatus(status) searchByTitle(kw) findByJobType(type) findByDivision(id)
JobApplicationApiService	/job-applications	getAll() getById() save() update() delete()	findByJobId(jobId) findByApplicantId(id) findByStatus(status)
JobAlertApiService	/job-alerts	getAll() getById() save() update() delete()	findById(userId) findActive()
HiringPipelineApiService	/hiring-pipeline	getAll() getById() save() update() delete()	findByJobId(jobId) findByStage(stage)
InterviewRoundApiService	/interview-rounds	getAll() getById() save() update() delete()	findByApplicationId(id) findUpcoming()
ResumeApiService	/resumes	getAll() getById() save() update() delete()	findById(userId) findDefault(userId) findPublic()
EducationApiService	/educations	getAll() getById() save() update() delete()	findByResumeId(resumeId)
ExperienceApiService	/experiences	getAll() getById() save() update() delete()	findByResumeId(resumeId)
CertificationApiService	/certifications	getAll() getById() save() update() delete()	findByResumeId(resumeId)
ProjectApiService	/projects	getAll() getById() save() update() delete()	findByResumeId(resumeId)
GigApiService	/gigs	getAll() getById() save() update() delete()	findByFreelancerId(id) findByCategoryId(id) findByStatus(status) searchByTitle(kw) findTopRated()
GigPackageApiService	/gig-packages	getAll() getById() save() update() delete()	findByGigId(gigId)

API Service	Endpoint	Standard 5 Methods	Additional Methods
GigCategoryApiService	/gig-categories	getAll() getById() save() update() delete()	findTopLevel() findByParentId(parentId)
OrderApiService	/orders	getAll() getById() save() update() delete()	findByClientId(id) findByFreelancerId(id) findByStatus(status) findByGigId(gigId)
DisputeApiService	/disputes	getAll() getById() save() update() delete()	findByOrderId(orderId) findByStatus(status) findOpen()
WalletApiService	/wallets	getAll() getById() save() update() delete()	findByUserId(userId)
TransactionApiService	/transactions	getAll() getById() save() update() delete()	findByWalletId(id) findByType(type) findByReferenceId(id)
MessageApiService	/messages	getAll() getById() save() update() delete()	findByOrderId(orderId) findUnread(receiverId) markRead(messageId)

15. Workflow Diagrams

How the major user journeys flow through the system

Job Application Workflow

1. Browse	Guest browses /jobs page
2. Login	JOBSEEKER logs in — authGuard redirects if not logged in
3. Apply	Fills cover letter form, selects resume → JobApplicationService.save()
4. Status	Application status: APPLIED → SHORTLISTED → INTERVIEW_SCHEDULED
5. Interview	Employer creates InterviewRound — notification sent to jobseeker
6. Decision	Employer marks HIRED or REJECTED — final notification triggered

Freelance Order Workflow

1. Browse	Guest browses /gigs — views packages and pricing
2. Order	CLIENT selects package, writes requirements → OrderService.save()
3. Escrow	Client wallet.balance decreases. wallet.escrowBalance increases.
4. Deliver	FREELANCER works, clicks Deliver → order.status = DELIVERED
5. Review	Client reviews delivery — accepts or requests revision
6. Complete	On accept: escrow released → freelancer wallet.balance increases

Auth & OTP Workflow

1. Register	POST /users — account created with status PENDING_VERIFICATION
2. OTP sent	OtpModel created — 6-digit code sent to email/phone
3. Verify	User enters code → OtpService validates → isUsed = true
4. Verified	user.isEmailVerified = true → status = ACTIVE → redirect to dashboard
5. Reset	Forgot password → new OtpModel with purpose PASSWORD_RESET
6. New pass	OTP validated → PATCH /users/:id with new passwordHash

16. Database Design — MySQL Tables

All 26 entities — normalized — Spring Boot ready

Every model maps to exactly one MySQL table. Foreign key columns use VARCHAR(36) for UUID compatibility. JSON arrays (skills, tags, features) use JSON column type in MySQL 8+.

TABLE: users

id VARCHAR(36) PK full_name VARCHAR(255) email VARCHAR(255) UNIQUE phone VARCHAR(20) password_hash VARCHAR(255) roles JSON profile_picture TEXT NULL division_id VARCHAR(36) district_id VARCHAR(36) upazila_id VARCHAR(36) post_code VARCHAR(10) address_line TEXT status ENUM(ACTIVE,SUSPENDED,PENDING_VERIFICATION) is_email_verified BOOLEAN is_phone_verified BOOLEAN created_at DATETIME updated_at DATETIME

TABLE: otps

id VARCHAR(36) PK user_id VARCHAR(36) FK→users code VARCHAR(6) purpose ENUM(EMAIL_VERIFY,PHONE_VERIFY,PASSWORD_RESET) expires_at DATETIME is_used BOOLEAN created_at DATETIME

TABLE: notifications

id VARCHAR(36) PK user_id VARCHAR(36) FK→users type VARCHAR(50) title VARCHAR(255) body TEXT is_read BOOLEAN DEFAULT false reference_id VARCHAR(36) NULL created_at DATETIME

TABLE: ratings

id VARCHAR(36) PK order_id VARCHAR(36) FK→orders reviewer_id VARCHAR(36) FK→users reviewee_id VARCHAR(36) FK→users target_type ENUM(FREELANCER,CLIENT,EMPLOYER) score TINYINT(1-5) comment TEXT created_at DATETIME

TABLE: jobs

id VARCHAR(36) PK employer_id VARCHAR(36) FK→users title VARCHAR(255) description TEXT requirements TEXT responsibilities TEXT job_type ENUM(FULL_TIME,PART_TIME,CONTRACT,INTERNSHIP) experience_level ENUM(ENTRY,MID,SENIOR,EXECUTIVE) salary_min DECIMAL(12,2) NULL salary_max DECIMAL(12,2) NULL is_salary_negotiable BOOLEAN skills JSON division_id VARCHAR(36) district_id VARCHAR(36) upazila_id VARCHAR(36) post_code VARCHAR(10) address_line TEXT deadline DATE status ENUM(DRAFT,ACTIVE,CLOSED,ARCHIVED) total_applications INT DEFAULT 0 created_at DATETIME updated_at DATETIME

TABLE: job_applications

id VARCHAR(36) PK job_id VARCHAR(36) FK→jobs applicant_id VARCHAR(36) FK→users resume_id VARCHAR(36) FK→resumes NULL cover_letter TEXT status ENUM(APPLIED,SHORTLISTED,INTERVIEW_SCHEDULED,REJECTED,HIRED,WITHDRAWN) applied_at DATETIME updated_at DATETIME

TABLE: job_alerts

id VARCHAR(36) PK user_id VARCHAR(36) FK→users keyword VARCHAR(255) job_type VARCHAR(50) NULL experience_level VARCHAR(50) NULL division_id VARCHAR(36) NULL is_active BOOLEAN DEFAULT true created_at DATETIME

TABLE: hiring_pipeline

id VARCHAR(36) PK job_id VARCHAR(36) FK→jobs applicant_id VARCHAR(36) FK→users application_id VARCHAR(36) FK→job_applications stage ENUM(APPLIED,SCREENING,INTERVIEW,OFFER,HIRED,REJECTED) updated_at DATETIME

TABLE: interview_rounds

id VARCHAR(36) PK application_id VARCHAR(36) FK→job_applications round TINYINT type ENUM(PHONE,VIDEO,ONSITE,TECHNICAL,HR) scheduled_at DATETIME location TEXT NULL meeting_link TEXT NULL outcome ENUM(PASSED,FAILED,RESCHEDULED) NULL notes TEXT NULL created_at DATETIME

TABLE: resumes

id VARCHAR(36) PK user_id VARCHAR(36) FK→users headline VARCHAR(255) summary TEXT career_objective TEXT skills JSON available_for VARCHAR(50) expected_salary DECIMAL(12,2) present_salary DECIMAL(12,2) career_level VARCHAR(50) division_id VARCHAR(36) district_id VARCHAR(36) upazila_id VARCHAR(36) post_code VARCHAR(10) address_line TEXT permanent_address TEXT NULL github_url TEXT NULL portfolio_url TEXT NULL linkedin_url TEXT NULL cv_visibility ENUM(PUBLIC,PRIVATE,RECRUITERS_ONLY) profile_completion TINYINT is_default BOOLEAN created_at DATETIME updated_at DATETIME

TABLE: educations

id VARCHAR(36) PK resume_id VARCHAR(36) FK→resumes institution VARCHAR(255) degree VARCHAR(100) field_of_study VARCHAR(100) board VARCHAR(100) NULL start_year YEAR end_year YEAR NULL result VARCHAR(20) NULL achievements TEXT NULL

TABLE: experiences

id VARCHAR(36) PK resume_id VARCHAR(36) FK→resumes company_name VARCHAR(255) job_title VARCHAR(255) department VARCHAR(100) NULL start_date DATE end_date DATE NULL is_current_job BOOLEAN responsibilities TEXT achievements TEXT NULL

TABLE: certifications

id VARCHAR(36) PK resume_id VARCHAR(36) FK→resumes name VARCHAR(255) issuing_organization VARCHAR(255) issue_date DATE expiry_date DATE NULL credential_id VARCHAR(100) NULL credential_url TEXT NULL

TABLE: projects

id VARCHAR(36) PK resume_id VARCHAR(36) FK→resumes project_name VARCHAR(255) description TEXT technologies JSON github_url TEXT NULL live_url TEXT NULL start_date DATE NULL end_date DATE NULL

TABLE: gig_categories

id VARCHAR(36) PK name VARCHAR(100) slug VARCHAR(100) UNIQUE parent_id VARCHAR(36) FK→gig_categories NULL icon VARCHAR(50) NULL

TABLE: gigs

id VARCHAR(36) PK freelancer_id VARCHAR(36) FK→users category_id VARCHAR(36) FK→gig_categories title VARCHAR(255) description TEXT tags JSON cover_image TEXT NULL images JSON status ENUM(ACTIVE,PAUSED,PENDING_REVIEW,REJECTED) average_rating DECIMAL(2,1) DEFAULT 0 total_orders INT DEFAULT 0 created_at DATETIME updated_at DATETIME

TABLE: gig_packages

id VARCHAR(36) PK gig_id VARCHAR(36) FK→gigs name ENUM(BASIC,STANDARD,PREMIUM) description TEXT price DECIMAL(10,2) delivery_days TINYINT revisions TINYINT features JSON

TABLE: orders

id VARCHAR(36) PK gig_id VARCHAR(36) FK→gigs client_id VARCHAR(36) FK→users freelancer_id VARCHAR(36) FK→users package_id VARCHAR(36) FK→gig_packages requirements TEXT price DECIMAL(10,2) platform_fee DECIMAL(10,2) freelancer_earning DECIMAL(10,2) status ENUM(PENDING,IN_PROGRESS,DELIVERED,REVISION_REQUESTED,COMPLETED,CANCELLED,DISPUTED) delivery_deadline DATETIME delivered_at DATETIME NULL completed_at DATETIME NULL revisions_used TINYINT DEFAULT 0 created_at DATETIME updated_at DATETIME

TABLE: disputes

id VARCHAR(36) PK order_id VARCHAR(36) FK→orders UNIQUE raised_by_id VARCHAR(36) FK→users reason ENUM(NO_DELIVERY,POOR_QUALITY,NOT_AS_DESCRIBED,COMMUNICATION,OTHER) description TEXT status ENUM(OPEN,UNDER_REVIEW,RESOLVED,CLOSED) admin_note TEXT NULL resolved_at DATETIME NULL created_at DATETIME

TABLE: wallets

id VARCHAR(36) PK user_id VARCHAR(36) FK→users UNIQUE balance DECIMAL(12,2) DEFAULT 0 escrow_balance DECIMAL(12,2) DEFAULT 0 currency VARCHAR(3) DEFAULT 'BDT' updated_at DATETIME

TABLE: transactions

id VARCHAR(36) PK wallet_id VARCHAR(36) FK→wallets type ENUM(DEPOSIT,WITHDRAWAL,ORDER_PAYMENT,ORDER_REFUND,PLATFORM_FEE,FREELANCER_EARNING,ESCROW_LOCK,ESCROW_RELEASE) amount DECIMAL(12,2) reference_id VARCHAR(36) NULL description VARCHAR(255) created_at DATETIME

TABLE: messages

id VARCHAR(36) PK order_id VARCHAR(36) FK→orders sender_id VARCHAR(36) FK→users receiver_id VARCHAR(36) FK→users content TEXT attachment_url TEXT NULL attachment_type ENUM(IMAGE,PDF,ZIP,OTHER) NULL is_read BOOLEAN DEFAULT false sent_at DATETIME

TABLE: reports

id VARCHAR(36) PK reporter_id VARCHAR(36) FK→users target_id VARCHAR(36) target_type ENUM(USER, JOB, GIG) reason
ENUM(SPAM, FRAUD, INAPPROPRIATE, FAKE, OTHER) description TEXT status ENUM(PENDING, REVIEWED, ACTION_TAKEN, DISMISSED)
admin_note TEXT NULL created_at DATETIME

17. JSON Server — db.json Reference

All collections for the fake backend

db.json must have all these top-level keys. JSON Server auto-generates CRUD endpoints for each. Run with:
json-server --watch db.json --port 3000

db.json Key	REST Endpoint	Purpose
users	/users	All user accounts
otps	/otps	OTP verification codes
password_resets	/password_resets	Password reset tokens
notifications	/notifications	In-app notifications
ratings	/ratings	Reviews and star ratings
reports	/reports	User/content reports
jobs	/jobs	Job listings
job_applications	/job_applications	Job applications
job_alerts	/job_alerts	Saved job search alerts
hiring_pipeline	/hiring_pipeline	Hiring stage per applicant
interview_rounds	/interview_rounds	Interview schedule
resumes	/resumes	CV root records
educations	/educations	Education entries
experiences	/experiences	Work experience entries
certifications	/certifications	Certification entries
projects	/projects	Portfolio project entries
gig_categories	/gig_categories	Gig category lookup
gigs	/gigs	Freelance service listings
gig_packages	/gig_packages	Gig pricing tiers
orders	/orders	Placed orders
disputes	/disputes	Order disputes
wallets	/wallets	User wallets
transactions	/transactions	All money movements
messages	/messages	Order chat messages

18. Environment & Migration Plan

The one file that controls your entire backend

This is the entire Spring Boot migration strategy. Change apiUrl in environment.ts. Every service reads from it. Nothing else changes in the frontend.

```
// src/environments/environment.ts
// TODAY — JSON Server
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000'
};

// FUTURE — Spring Boot (just uncomment, comment out the line above)
// export const environment = {
//   production: true,
//   apiUrl: 'https://api.yourplatform.com/api'
// };
```

How every API service uses it

```
import { environment } from '../../../environments/environment';

@Injectable({ providedIn: 'root' })
export class JobApiService {
  private url = `${environment.apiUrl}/jobs`; // reads from environment
  ...
}
```

Migration checklist

Step	Action	Status
1	Change apiUrl in environment.ts to Spring Boot URL	Day of migration
2	Add Authorization header in jwt.interceptor.ts	Already prepared
3	Spring Boot returns same JSON structure as db.json	Backend team responsibility
4	All endpoints follow same path: /users, /jobs, /gigs etc.	Agreed in design phase
5	JWT token stored in TokenService — not localStorage directly	Already in core/
6	Auth endpoints: POST /auth/login, POST /auth/register	Backend to implement
7	Test each feature module independently	QA plan

19. Build Roadmap — 7 Phases

Sequential — do not skip phases

Phase 1 Foundation

- ✓ New Angular project with strict mode
- ✓ Install Bootstrap 5 + bootstrap-icons
- ✓ Create environments/environment.ts
- ✓ Create all feature folder structure
- ✓ Create 3 layouts: public, dashboard, admin
- ✓ Create shared components: navbar, sidebar, footer, star-rating, status-badge, otp-input
- ✓ Create core: BaseApiService, JwtInterceptor, authGuard, adminGuard, roleGuard
- ✓ Create app.routes.ts with lazy-loaded shell
- ✓ Set up db.json with all 24 empty collections

Phase 2 All 26 Models

- ✓ Write every model file in the correct feature folder
- ✓ All ids are id?: string — optional, never required
- ✓ All foreign keys are required string fields
- ✓ AddressModel used in UserModel and JobModel — no plain strings
- ✓ ResumeModel has only array fields — no duplicate string fields
- ✓ One interface per file — no type aliases

Phase 3 All Services (2 per model)

- ✓ JobApiService + JobService — repeat for all 26 models
- ✓ AuthService: login, register, logout, session — no HttpClient
- ✓ AuthApiService: all HTTP calls
- ✓ All API services read from environment.apiUrl
- ✓ All filter methods added to API services
- ✓ All business methods added to Feature services

Phase 4 Auth Feature

- ✓ Login component with reactive form
- ✓ Register component — role selection at signup
- ✓ OTP verify component — reuse otp-input shared component
- ✓ Forgot password component
- ✓ Reset password component

- ✓ Session management via TokenService

- ✓ Guards working and tested

Phase 5 Job Portal

- ✓ Public job listing page with search + filter
- ✓ Job detail page (separate from gig detail)
- ✓ Employer: post job, view applications, manage pipeline
- ✓ Jobseeker: apply, track application status, manage job alerts
- ✓ CV builder: 5 sub-components (education, experience, certs, projects, skills)
- ✓ Interview scheduling for employers

Phase 6 Freelance Marketplace

- ✓ Public gig listing page with search + filter by category
- ✓ Gig detail page with 3-tier package selector
- ✓ Freelancer: create gig, manage packages, view orders, deliver work
- ✓ Client: browse gigs, place order, provide requirements
- ✓ Order tracking dashboard for both parties
- ✓ Dispute form for problematic orders

Phase 7 Payments + Admin + Chat

- ✓ Wallet component: balance, escrow, transaction history
- ✓ Withdrawal request form
- ✓ Order-based chat window with file attachment support
- ✓ Admin: user management table (list, suspend, ban)
- ✓ Admin: job and gig moderation queue
- ✓ Admin: dispute resolution panel
- ✓ Admin: analytics dashboard

20. Risk Analysis

What to watch for as the project grows

Severity	Risk	Detail
HIGH	Hardcoded URLs creep back in	Any developer who writes localhost:3000 directly in a service bypasses environment.ts. Code review must catch this. Add an ESLint rule to flag localhost strings.
HIGH	Business logic moves into components	As features grow, developers take shortcuts and put API calls or if/else logic inside templates. Enforce the 3-layer rule in every code review. Components may only call service methods.
HIGH	ngModel sneaks into forms	Template-driven forms are faster to write but incompatible with reactive validation. Disable FormsModule in all feature modules. Import only ReactiveFormsModule.
MEDIUM	Model mismatch with Spring Boot	If the backend team uses snake_case (user_id) and Angular models use camelCase (userId), you will get undefined values everywhere. Decide on camelCase everywhere and add a Spring Boot HttpMessageConverter to serialize accordingly.
MEDIUM	Address model bypassed	Developers may add a plain city string field for convenience. Enforce AddressModel usage in all models through code review. Delete any plain city/area/district string fields found in models.
MEDIUM	Profile completion calculation drifts	ResumeService.calculateProfileCompletion() must stay in sync with the fields in ResumeModel. When a new field is added to the model, the calculation must be updated.
LOW	db.json grows too large for JSON Server	JSON Server performance degrades above ~5000 records. For development this is fine. If you need more data, switch to Spring Boot early.
LOW	Chat requires WebSocket in production	The current HTTP polling design works for dev but is not production-grade. Spring Boot WebSocket (STOMP) will replace polling when you migrate. The MessageModel does not need to change.